

LUMEN ANALYTICS

A Data Analyst / BI Analyst Mentorship Project
Project Brief & Question Bank

Built on PostgreSQL | Companion to the DataCamp Associate Data Analyst in SQL track

Why B2B SaaS

Of the realistic candidate domains, a subscription software business gives the widest coverage per table. A single, connected customer base naturally produces recurring financial transactions (finance analysis), acquisition and channel data (marketing analysis), behavioral usage data (product analysis), and support load (operations analysis) — all time-stamped, all joinable back to one parent entity. Churn, monthly recurring revenue, cohort retention, and feature adoption are also among the most common scenarios asked about in real Data Analyst and BI Analyst interviews, so the skills built here transfer directly, regardless of what industry you end up working in.

Company Background

Lumen Analytics is a fictional B2B SaaS company that sells a dashboarding and reporting tool to other companies. Customers sign up for one of three plans — **Starter**, **Growth**, or **Scale** — and are billed roughly monthly. Customers come from five regions and a range of company sizes, and are acquired through six different channels (organic search, paid ads, referral, outbound sales, content marketing, and partner deals). Some customers churn; most don't — yet.

You've joined Lumen's small analytics function. Requests will arrive from Finance, Marketing, Product, Operations, Customer Success, and company leadership, the way they actually would at a real company: as questions, not SQL syntax instructions. Deciding *how* to answer each one is your job.

Dataset Overview

Four related tables, all built around one parent entity: the customer.

Table	Grain (one row =)	Primary Key	Foreign Key
customers	one subscribing company	customer_id	(parent table)
payments	one billing transaction	payment_id	customer_id
usage_events	one product interaction	event_id	customer_id
support_tickets	one support ticket	ticket_id	customer_id

Relationships: *customers* is the parent table. *payments*, *usage_events*, and *support_tickets* are each many-to-one against it — every customer can have zero, one, or many rows in each child table. This shape is deliberate: once you've internalized "check what a join does to row counts before joining two child tables together," the same instinct applies uniformly across the whole project, and later, to any real company's data.

Known data-quality issues, planted on purpose: a handful of missing *acquisition_channel* values, a couple of *support_tickets* rows where *closed_date* is earlier than *opened_date*, a few *category* values in inconsistent casing, and a small number of duplicate *customers* rows (the same company appearing twice under different IDs). Full column-level definitions are in the README sheet of the accompanying Excel workbook.

Learning Progression

The project moves through 20 phases, each mapped to a stage of the DataCamp Associate Data Analyst in SQL track plus the professional skills that sit beyond it. Work through them in order — later phases assume the judgment built in earlier ones (correct grain, correct denominator, NULL-awareness) without re-explaining it.

- 1. Data Familiarization & Grain
- 2. NULL Handling & Data Quality
- 3. Text & Numeric Functions
- 4. Date/Time Analysis
- 5. Aggregation & KPIs
- 6. Descriptive Statistics & Percentiles
- 7. Joins & Data Grain (multi-table)
- 8. Exploratory Data Analysis (structured process)
- 9. Subqueries & CTEs
- 10. Window Functions I -- Ranking
- 11. Window Functions II -- LAG/LEAD & Sequences
- 12. Window Functions III -- Running & Rolling Metrics
- 13. Cohort & Retention Analysis
- 14. Customer Analytics
- 15. Marketing Analytics
- 16. Finance & Profitability Analytics
- 17. Product Analytics
- 18. Operations Analytics
- 19. KPI Design & Mini Investigations
- 20. Executive Analysis & Final Capstone

How to Use This Question Bank

Each phase opens with a short concept note, then questions organized across up to five levels of difficulty:

- **Level 1 -- Foundation:** simple exercises to confirm you understand the mechanics.
- **Level 2 -- Application:** the same concept, applied against Lumen's real data.
- **Level 3 -- Integration:** combined with techniques from earlier phases.
- **Level 4 -- Workplace Request:** a stakeholder question where you decide which technique fits.
- **Level 5 -- Investigation:** open-ended, possibly ambiguous, with no single correct query.

No solutions are included by design. Work through a phase, then bring your queries back for review the way you would in a real code review: correctness, handling of NULLs and grain, readability, and whether your conclusion actually follows from your result.

Question Bank

Phase 1 -- Data Familiarization & Grain

Concept: before answering anyone's question, confirm what you're looking at. Row counts, uniqueness of primary keys, and a first look at each table's shape.

Scenario: *You've just joined Lumen's analytics team. Before anyone lets you near a report, confirm you understand the data you've inherited.*

Level 1. For each of the 4 tables, write a query confirming its primary key is actually unique (no duplicate IDs).

Level 1. Return the row count of each table, and the earliest and latest date present in each date column across the dataset.

Level 2. List the distinct values in customers.region, customers.plan_type_at_signup, and payments.payment_status. Do any look suspicious?

Level 3. For every customer_id in payments, usage_events, and support_tickets, confirm it exists in customers. Are there any orphaned rows?

Level 4. Your manager asks: "Give me a one-paragraph summary of what this dataset can and can't tell us." Write that paragraph, grounded in what you've actually found so far.

Phase 2 -- NULL Handling & Data Quality

Concept: NULL is not zero, not blank, and not "unknown" in a way SQL treats consistently by default. COALESCE, IS NULL / IS NOT NULL, and the effect of NULLs inside aggregates and joins.

Scenario: *Marketing is about to run a report crediting acquisition channels for revenue. Before they do, they want to know how much of the data is actually usable.*

Level 1. Count how many customers rows have a NULL acquisition_channel.

Level 2. Recompute that count as a percentage of all customers. Then write a version of the channel breakdown that labels NULLs as 'Unknown' instead of dropping them silently -- using COALESCE.

Level 3. Find every support_tickets row where closed_date is earlier than opened_date. Is this a data entry error, or could there be a legitimate explanation? Justify your answer with a follow-up query if needed.

Level 4. The VP of Customer Success asks: "What % of our tickets are still unresolved?" Decide what 'unresolved' should mean given the resolution_status values available, and justify your definition before answering.

Level 5. Investigate: are the duplicate customer records (the same company appearing under two different customer_id values) inflating any metric you've calculated so far? Pick one metric and check.

Phase 3 -- Text & Numeric Functions

Concept: UPPER/LOWER/INITCAP, string concatenation, ROUND/TRUNC, CASE for recoding, and normalizing messy categorical values.

Scenario: Operations flags that the `support_tickets.category` field looks inconsistent when they filter their dashboard.

Level 1. Write a query that shows each distinct category value in `support_tickets` exactly as stored, with a count of rows for each.

Level 2. Rewrite that query so casing variants of the same category (e.g., 'Bug Report' and 'BUG REPORT') are collapsed into one consistent value.

Level 3. Using CASE, bucket `customers.company_size_band` into two groups: 'SMB' (1-10, 11-50) and 'Mid-Market+' (51-200, 201-500, 500+). Count customers in each bucket.

Level 4. Finance wants every payment amount displayed as a clean 2-decimal currency figure in a report, with refunds clearly distinguishable from regular charges. Design the output.

Phase 4 -- Date/Time Analysis

Concept: `DATE_TRUNC`, `EXTRACT/DATE_PART`, date arithmetic and intervals, `AGE()`, and building complete time series with `generate_series` to expose gaps that a plain `GROUP BY` would hide.

Scenario: The CFO wants a monthly revenue trend and doesn't want any month silently missing just because nothing happened to fall into it.

Level 1. Truncate `payments.payment_date` to the month using `DATE_TRUNC`, and count payments per month.

Level 2. Sum `payments.amount` (excluding refunds/failed payments) by month. Order chronologically.

Level 3. For each customer, calculate their tenure in days as of today (or as of their `churn_date` if they've churned) using `AGE()` or date subtraction.

Level 4. The CFO's actual concern, restated: "Are there any months where we had zero recorded payments at all?" Your Level 2 query would simply omit those months rather than show them as zero. Use `generate_series` to build a complete monthly calendar and `LEFT JOIN` your payment totals onto it so missing months show explicitly as 0.

Level 5. Investigate: is there a seasonal pattern in either new signups (`customers.signup_date`) or churn (`customers.churn_date`)? State your finding and how confident you are in it given the data volume.

Phase 5 -- Aggregation & KPIs

Concept: `COUNT` vs `COUNT(DISTINCT ...)`, conditional aggregation with `CASE WHEN` inside `COUNT/SUM`, `GROUP BY` vs `HAVING`, and being explicit about denominators.

Scenario: The Head of Sales wants a simple scorecard: how many customers do we have, broken down by plan and region, and how much are they worth?

Level 1. Count active customers (`churn_date IS NULL`) by `plan_type_at_signup`.

Level 2. For each region, calculate total payments received (successful payments only) and the number of distinct paying customers.

Level 3. Calculate an average revenue per customer figure by region. Be explicit in your query and your explanation about what's in your denominator -- all customers, or only paying ones?

Level 4. Sales asks: "Which regions have more than 10 active customers?" -- a HAVING-filter question. Answer it, then tell Sales in one sentence what the number doesn't tell them.

Phase 6 -- Descriptive Statistics & Percentiles

Concept: AVG vs MEDIAN (PERCENTILE_CONT), STDDEV/VARIANCE, quartiles, and why a mean can mislead on skewed data -- directly reinforcing your Introduction to Statistics course.

***Scenario:** Finance is worried the average payment figure you gave them last phase might be skewed by a small number of large Scale-plan customers.*

Level 1. Calculate the average and the median (PERCENTILE_CONT(0.5)) payment amount. Are they close, or far apart?

Level 2. Calculate the standard deviation of payment amounts. What does a large standard deviation relative to the mean suggest about the distribution?

Level 3. Break customers into quartiles by their total lifetime payments using NTILE(4). Which quartile of customers is generating the majority of revenue?

Level 4. Finance's actual question: "Is 'average revenue per customer' even a useful number to report given what you've found?" Write your recommendation, with the numbers to back it up.

Phase 7 -- Joins & Data Grain (Multi-table)

Concept: before writing any join spanning more than 2 tables, identify the grain of each table and what happens to row counts when they're combined. This is the single most important habit in this whole project.

***Scenario:** Customer Success wants one row per customer showing their plan, total payments, number of support tickets, and number of usage events -- all in one view.*

Level 1. Join customers to payments and count total rows. Does the count match customers, payments, or neither? Explain why.

Level 2. Before writing any query: if you joined payments AND usage_events onto customers in the same query (not pre-aggregated), what would happen to a customer's payment totals? Answer in words first, then prove it by writing the query and comparing the wrong total to the correct one.

Level 3. Now build the correct version: pre-aggregate payments and usage_events into per-customer summaries separately (using CTEs), then join those summaries onto customers -- avoiding the fan-out from Level 2.

Level 4. Customer Success's actual request, in full: one row per customer with plan_type_at_signup, lifetime successful payment total, ticket count, and usage event count, sorted by ticket count descending (their highest-friction customers first).

Level 5. Investigate: is there a relationship between how many support tickets a customer has filed and whether they've churned? State your finding and at least one alternative explanation for it that isn't "tickets cause churn."

Phase 8 -- Exploratory Data Analysis (Structured Process)

Concept: a repeatable EDA process -- (1) understand what each table represents, (2) profile row counts and distributions, (3) assess quality, (4) explore individual variables, (5) explore relationships between variables,

(6) segment, (7) investigate anomalies, (8) form hypotheses, (9) test them with SQL, (10) communicate findings in plain language.

Scenario: *You've been asked to produce a short "state of the customer base" writeup for a new VP joining next month, who has never seen this data before.*

Level 3. Run through steps 1-6 of the EDA process above against the customers and payments tables. Keep brief notes at each step -- this is as much about the process as any single query.

Level 4. Pick one anomaly or surprising pattern you noticed during your EDA pass. Form a hypothesis for why it exists, then write a SQL query that would support or refute that hypothesis.

Level 5. Write the actual one-page summary for the incoming VP: what Lumen's customer base looks like, the 2-3 most important patterns you found, and one open question you'd want to investigate further with more data than this table currently has.

Phase 9 -- Subqueries & CTEs

Concept: scalar subqueries, subqueries in WHERE and FROM, correlated subqueries, and multi-step CTEs for readable, layered analysis. When a CTE earns its place versus when it's just decoration.

Scenario: *The Head of Product wants to know which customers are using the product far more, or far less, than typical for their plan tier.*

Level 1. Using a subquery, find all customers whose total successful payments exceed the average total payment across all customers.

Level 2. Using a CTE, calculate each customer's usage_event count, then in the final SELECT flag customers whose count is in the bottom 10% (hint: you'll want a percentile or NTILE from Phase 6 here).

Level 3. Build a 2-stage CTE: first stage aggregates usage_events per customer per plan tier; second stage compares each customer's usage to their plan tier's average usage, showing customers meaningfully above or below their peers.

Level 4. Product's actual ask: "Give me the 10 customers using the product the least, relative to how long they've been a customer." You'll need to decide what 'relative to tenure' should mean mathematically -- state your definition before writing the query.

Phase 10 -- Window Functions I: Ranking

Concept: OVER(), PARTITION BY, ORDER BY inside a window, ROW_NUMBER, RANK, DENSE_RANK, and NTILE. When to reach for a window function instead of GROUP BY -- the key distinction being whether you need to keep every row, or collapse to one row per group.

Scenario: *Sales wants a leaderboard: the top 3 customers by lifetime payment value, within each region.*

Level 1. Using ROW_NUMBER(), rank all customers by total successful payments, highest first, across the whole company (no partitioning yet).

Level 2. Now PARTITION BY region so the ranking restarts for each region.

Level 3. Explain, in your own words, what would change in your result if you used RANK() instead of ROW_NUMBER() -- then find (or construct a case for) a tie in the data that would actually show the difference.

Level 4. Sales's actual request: the top 3 customers by lifetime value in each region, in one result set, with their rank clearly shown. Filter to just the top 3 per region using a CTE plus a WHERE on the window function's result (remember: window functions can't be filtered directly in WHERE in the same query level).

Phase 11 -- Window Functions II: LAG/LEAD & Sequences

Concept: comparing a row to the row before or after it within an ordered partition -- essential for month-over-month change, time-between-events, and detecting first/last occurrences.

Scenario: *The CFO wants to know not just what monthly revenue is, but whether it's accelerating, flat, or declining.*

Level 1. Using your Phase 4 monthly revenue totals as a base, use LAG() to add a column showing the previous month's total next to each row.

Level 2. Calculate month-over-month percentage change in revenue using the LAG() column you just built.

Level 3. For each customer, use LAG() partitioned by customer_id to calculate the number of days between their consecutive payments. Are there customers with unusually long gaps between payments?

Level 4. The CFO's actual concern: "Tell me when our growth started slowing down, not just that it did." Use your month-over-month figures to identify the specific period(s) where momentum shifted, and explain your reasoning.

Phase 12 -- Window Functions III: Running & Rolling Metrics

Concept: running totals and cumulative metrics using a frame clause (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), moving averages over a rolling window, and percent-of-total calculations.

Scenario: *The Board wants to see cumulative revenue growth over the company's history, plus a smoothed trend line that isn't noisy month to month.*

Level 1. Calculate a running total of monthly revenue, ordered chronologically.

Level 2. Calculate a 3-month moving average of monthly revenue.

Level 3. For each plan tier, calculate what percentage of total company revenue it represents, using a window function rather than a second query.

Level 4. Build the Board-ready output: month, monthly revenue, running total, 3-month moving average, and month-over-month % change (from Phase 11), all in one result set.

Phase 13 -- Cohort & Retention Analysis

Concept: grouping customers into signup cohorts (e.g., by signup month), then tracking what percentage of each cohort is still active N months later -- the backbone of subscription-business analytics.

Scenario: *The CEO wants to know if newer cohorts of customers are stickier than older ones, or if retention has been getting worse.*

Level 2. Group customers into monthly signup cohorts using DATE_TRUNC on signup_date. Count customers per cohort.

Level 3. For each cohort, calculate how many customers are still active (churn_date IS NULL) as of today, and what percentage that represents.

Level 4. Build a simplified retention view: for each signup cohort, what percentage of customers were still active at the 3-month mark and the 6-month mark after signup (using churn_date to determine this, not today's date)?

Level 5. The CEO's actual question: "Is retention improving or getting worse over time?" Compare early cohorts to recent cohorts and present your finding, including any caveats about cohorts that haven't had enough time to prove themselves yet.

Phase 14 -- Customer Analytics

Concept: combining everything so far into customer-level analysis -- segmentation, value tiers, and behavior-based grouping.

***Scenario:** Customer Success wants to build a simple health score to prioritize who they reach out to this week.*

Level 3. For each active customer, combine: tenure in days, total payments, ticket count, and usage event count into one row.

Level 4. Using the combined view above, segment customers into 'Healthy', 'At Risk', and 'Needs Attention' using your own reasonable thresholds on ticket volume and usage activity. State your thresholds explicitly and why you chose them.

Level 5. Investigate: pick your 'Needs Attention' segment and look for a common pattern among them -- shared region, plan, acquisition channel, or signup period. Is there a signal, or is it noise given the sample size?

Phase 15 -- Marketing Analytics

Concept: acquisition channel performance -- not just volume, but quality (does the channel bring customers who pay and stay?).

***Scenario:** The CMO wants to reallocate next quarter's budget and wants to know which channel is actually worth the spend.*

Level 2. Count customers acquired per channel, and their churn rate per channel.

Level 3. Calculate average lifetime payment value per customer, broken down by acquisition_channel. Handle the NULL channel values explicitly rather than letting them disappear.

Level 4. The CMO's actual request: "Rank our channels by customers acquired AND by revenue quality, and tell me if those rankings agree or disagree." Present both rankings side by side and explain any channel where volume and quality tell different stories.

Phase 16 -- Finance & Profitability Analytics

Concept: MRR-style thinking, refund/failure rates, and revenue concentration -- how dependent is the business on its largest customers?

Scenario: *The CFO is preparing a board deck and needs a clean revenue picture, refund exposure, and a concentration risk check.*

Level 2. Calculate the refund rate (refunded payments as a % of all payments) overall, and by plan_type.

Level 3. Calculate what percentage of total revenue comes from the top 10% of customers by payment value (tie this back to your Phase 6 quartile work).

Level 4. Build a simple monthly recurring revenue (MRR) proxy: for each month, sum successful payments from customers who have not yet churned as of that month. Explain any simplifying assumptions you had to make given the data available.

Level 5. The CFO's actual concern: "How exposed are we if we lost our single largest customer tomorrow?" Quantify it.

Phase 17 -- Product Analytics

Concept: feature-level usage analysis, adoption over time, and connecting usage patterns to plan tier and churn.

Scenario: *The Head of Product wants to know which features actually matter to retention, versus which ones nobody uses.*

Level 2. Count usage_events by feature_name (where populated) and by event_type overall.

Level 3. Compare feature usage patterns between churned and still-active customers. Do churned customers show a different usage mix before they left?

Level 4. Product's actual request: "Is there a specific feature that seems associated with customers sticking around longer?" Investigate and present your finding, being explicit about the difference between correlation and proof of causation here.

Phase 18 -- Operations Analytics

Concept: support ticket volume, resolution time, and whether operational friction concentrates in specific segments.

Scenario: *Operations leadership wants to know if their support load is growing faster than the customer base, and whether certain customer segments are disproportionately support-heavy.*

Level 2. Calculate average ticket resolution time in days (closed_date minus opened_date), excluding the bad rows you found back in Phase 2.

Level 3. Break resolution time down by priority. Are Urgent tickets actually resolved faster than Low priority ones, as you'd expect?

Level 4. Operations' actual request: "Are support tickets growing faster than our customer base?" Compare the growth rate of monthly ticket volume to the growth rate of the active customer count over the same period.

Phase 19 -- KPI Design & Mini Investigations

Concept: defining a KPI properly before calculating it -- numerator, denominator, grain, time period, filters, exclusions, NULL treatment, and business interpretation. By the end of this phase, produce a small KPI

dictionary, the kind a real BI team would maintain.

KPI Design Exercise

For each of the following KPIs, write out a full definition (numerator, denominator, grain, time window, exclusions, NULL handling) *before* writing any SQL, then implement it: Monthly Recurring Revenue, Customer Churn Rate, Average Revenue Per Account, Ticket Resolution Rate, Feature Adoption Rate.

Mini Investigations

Each of these is deliberately under-specified. Before writing SQL, write down what you'd need to clarify with the stakeholder, and the assumptions you're making in their absence.

Investigation 1. "Our newest customers don't seem to be sticking around. Can you look into it?"

Investigation 2. "Support costs feel like they're eating into our margins on the Starter plan. Is that actually true?"

Investigation 3. "Marketing thinks Referral is our best channel. Finance thinks it's Outbound Sales. Who's right?"

Investigation 4. "Explain why revenue went up last quarter -- was it new customers, existing customers spending more, or fewer refunds?"

Phase 20 -- Executive Analysis & Final Capstone

Concept: synthesizing everything into an executive-ready investigation. No hints, no suggested techniques -- decide what data you need, what quality checks matter, what the right metric and grain are, and how to present findings to someone who will never see your SQL.

The Capstone Brief

You've been asked to present to Lumen's leadership team next week. The prompt from the CEO is deliberately broad:

"We need to understand the overall health of the business right now — where we're growing, where we're at risk, and what you'd recommend we focus on next quarter. You have full access to our customer, payment, usage, and support data. Show me what you find."

Work through the full analyst process on your own:

- 1 Understand the problem and what "business health" should actually mean here.
- 2 Determine which tables and columns are relevant.
- 3 Assess data quality issues that could affect your conclusions.
- 4 Define the metrics you'll use, and why.
- 5 Explore the data broadly before narrowing in.
- 6 Form hypotheses about what's driving growth, risk, and opportunity.
- 7 Test those hypotheses with SQL.
- 8 Validate your findings -- check for fan-out, wrong denominators, or small-sample noise.
- 9 Identify your 3-5 key insights.
- 10 Write recommendations, not just observations.

- 11 Explicitly state the limitations of your analysis and what data you'd want but don't have.
- 12 Present it as you would to people who will never see your SQL -- plain language, the numbers that matter, and a clear "so what."

Appendix -- Importing Into PostgreSQL

You have two ways to get this data into PostgreSQL. Pick whichever fits how you're running Postgres locally.

Option A -- From the Excel workbook (as requested)

- 1 Open lumen_analytics_dataset.xlsx. Export each of the 4 data sheets (customers, payments, usage_events, support_tickets) as its own CSV file: File > Save As / Export > CSV, one file per sheet. Skip the README sheet -- it's documentation, not data.
- 2 Connect to your PostgreSQL database (psql, pgAdmin, DBeaver, or similar).
- 3 Create the 4 tables first, in this order, so the foreign keys resolve: customers, then payments / usage_events / support_tickets. Use the column names and types listed in the README sheet's data dictionary (SERIAL for each *_id primary key, DATE for date columns, NUMERIC(10,2) for amount, VARCHAR for text columns).
- 4 Load each CSV into its matching table. In psql, from the command line: \copy customers FROM 'customers.csv' WITH (FORMAT csv, HEADER true); -- repeat for the other 3 tables, loading customers first. In pgAdmin: right-click the table -> Import/Export Data -> point it at the matching CSV with 'Header' turned on.
- 5 Verify: SELECT COUNT(*) FROM customers; and repeat for the other 3 tables -- row counts should match what's in the README (73 / 1,076 / 713 / 125).

Option B -- Fastest path, if you still have it

If you kept the earlier .sql file generated for this same dataset, you can skip all of the above: open a connection to your database and run that script directly (psql -d your_database -f lumen_schema_and_data.sql, or paste it into pgAdmin's Query Tool and execute). It creates all 4 tables and loads every row in one step -- the CSV/Excel route above exists specifically to satisfy the Excel-first deliverable requested for this version of the project, and both paths land on identical data.

A note on the data types

date columns should be typed DATE, not TIMESTAMP -- there's no time-of-day component in this data. amount in payments is NUMERIC(10,2) and can legitimately be negative (refunds) -- don't add a CHECK constraint blocking negative values, or you'll block valid data on import.